

DISEÑO DE SISTEMAS DE INFORMACIÓN

“METAMAPA”

JUSTIFICACIONES DE DISEÑO

Trabajo Práctico Anual Integrador
-2025-

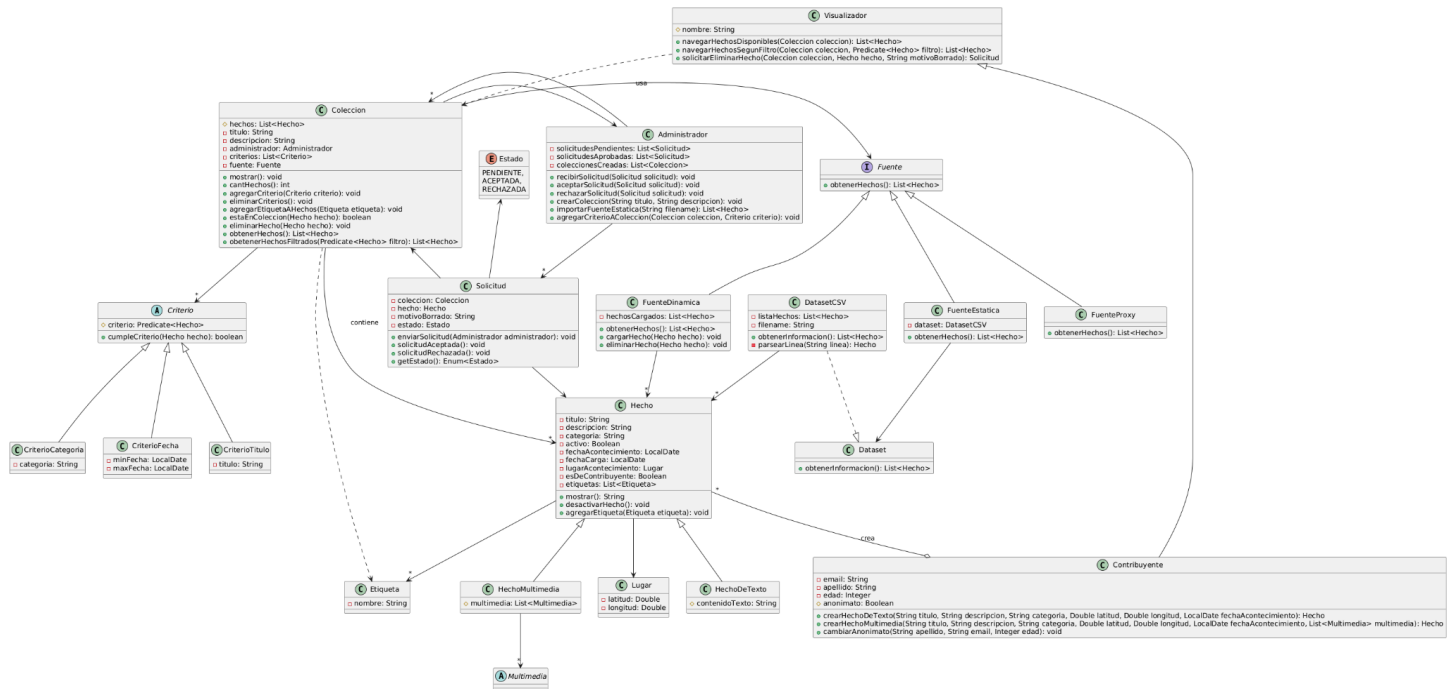
Grupo N°5



ENTREGA 1: Arquitectura y Modelado en Objetos - Parte I

Entregables

1. Modelo del Dominio: diagrama de clases inicial que contemple las funcionalidades requeridas.



- **Relaciones:**

Hablando un poco de las relaciones utilizamos la asociación simple dirigida para aquellos casos en los que una clase tiene como atributo un objeto de otra clase como pasa por ejemplo en la **Solicitud** que tiene un **Hecho** asociado, la Solicitud que tiene un Estado asociado o también la Colección que tiene un Administrador asociado. Cabe aclarar que en los casos especiales en donde ese atributo es una lista de objetos se agrega un asterisco como pasa por ejemplo en el **Dataset CSV** que tiene una lista de **Hechos**.

Sumado a esto, utilizamos relaciones de generalización para aquellos casos en los que una clase hereda a otra como pasa en **Visualizador** y **Contribuyente** o en los diferentes tipos de criterios.

Además, establecimos relaciones de uso en los casos que una clase necesita de otra para ofrecer una funcionalidad como cuando una clase recibe los parámetros de otra clase para realizar algo como pasa en la Colección con la Etiqueta o como hace el Contribuyente para crear el Hecho.

Por último, como tenemos una interface para representar la **Fuente** (clase padre) decidimos usar una relación de realización para los subtipos debido a que esta se utiliza para representar que una clase implementa una interface determinada.

2. Justificaciones de Diseño Iniciales.

- **Fuentes de datos:** orígenes de la información en el sistema, pudiendo ser estáticas (datasets), dinámicas (aportadas por usuarios) o intermediarias (**proxy**, integraciones con otras instancias de MetaMapa u otros sistemas).

Para esta entrega, las fuentes de datos se modelaron aplicando el **Patrón Template Method**. Si bien en esta entrega solo se modela el tipo de Fuente Estática se prevé que a futuro van a existir diferentes tipos de fuentes (como la dinámica o la proxy), pero todas con un mismo objetivo que es entregar información al sistema. Es por ello que se creó una clase abstracta **Fuente**, la cual define el esqueleto del algoritmo de entrega de datos definiendo los pasos que sí o sí se deben implementar. Las clases hijas implementan los pasos específicos según su tipo, manteniendo coherencia estructural y permitiendo el uso **polimórfico** de las diferentes fuentes. Además, este patrón facilita pruebas unitarias y mockeos en los tests al tener una clase base común.

Para **mayor cohesión**, **DatasetCSV**, es la clase que posee la lógica de lectura de archivos .csv a partir del nombre del archivo. Se logra evitar una '**God Class**', ya que la fuente estática solo conoce el nombre del archivo, encarga al dataset leerlo y extraer la información necesaria para crear los hechos. La interface **Dataset** se pensó por si en un futuro existe una nueva forma de conseguir los datos estáticos (CSV, Base de Datos, JSON, etc.). Posibilita que las fuentes estáticas cambien sus datasets de ser necesario.

- **Personas:** Contribuyentes, Administradores y Visualizadores. Ingresan a la plataforma para visualizar, modificar o utilizar la información disponible en el Sistema.

Sabemos que **Contribuyentes** y **Visualizadores** tienen un comportamiento en común: ambos pueden visualizar la información disponible según un filtro o también eliminar hechos. Por eso mismo, para evitar la repetición de código y promover la reutilización, se decidió que la clase Contribuyente herede de Visualizador estos métodos. De este modo, **Contribuyente** puede extender el comportamiento base definido en **Visualizador**, beneficiándose de los principios de **herencia** y **reutilización del código**. Ya que este, además de las funciones de Visualizador, puede crear hechos (ya sea de multimedia o de texto) y dejar de ser anónimo, es decir, agregarse un apellido, email y edad.

Por otra parte, para mantener un **bajo acoplamiento** y una arquitectura más flexible, se optó que la clase **Administrador** no herede ni de **Visualizador** ni de **Contribuyente**, ya que su rol y responsabilidades son distintas. En lugar de forzar relaciones jerárquicas innecesarias, se mantiene como una clase independiente, respetando el principio de

responsabilidad única y facilitando posibles cambios o extensiones futuras con posibles nuevas funcionalidades sin impacto en las otras clases.

- **Solicitudes de Eliminación:** Cualquier hecho del sistema debe admitir solicitudes de eliminación, para aquellas situaciones adecuadamente fundadas.

Sabemos que los usuarios pueden solicitar la eliminación de un hecho, mientras que los administradores son quienes reciben y evalúan estas solicitudes. Para lograr una **alta cohesión** y **evitar la sobrecarga de responsabilidades** en las clases **Administrador** o **Usuario (Contribuyente y Visualizador)**, se introdujo la clase **Solicitud**. Esta clase centraliza la lógica relacionada con la gestión de solicitudes, incluyendo el seguimiento de su estado y la decisión final sobre si un hecho debe eliminarse. De esta manera, se mejora la organización del sistema y se favorece su **mantenibilidad**. Esto nos deja la puerta abierta para agregar flujos más complejos como auditorías o notificaciones sin modificar la clase Usuario o Administrador. Sumado a esto, determinamos que el estado de una solicitud (aceptada, pendiente o rechazada) sea de tipo Enum para así poder evitar problemas de **consistencia de datos** con los diferentes textos u alternativas.

- **Hechos:** Hasta el momento, existen dos tipos de hechos: de texto y con soporte para contenido multimedia. Para esta primera iteración, solo se requerirá dar soporte al primero, pero es importante contemplarlo en el diseño de la plataforma.

Se prevé que a futuro van a existir diferentes tipos de hechos, pero todos con una misma funcionalidad. Se creó la superclase **Hecho**, la cual define el esqueleto. En este caso, las clases hijas implementan los pasos específicos según su tipo de contenido que puede ser texto o multimedia, manteniendo **coherencia estructural** y permitiendo el uso **polimórfico** para clases como el Hecho de texto y el hecho de multimedia suponiendo que tendrán diferentes funcionalidades. Cabe aclarar que todo esto lo hicimos para definir preventivamente el diseño sin saber cuáles serán los siguientes avances de la entrega 2. Por último, también agregamos una **data class Lugar** para juntar la longitud y la latitud en una sola clase, **evitando parámetros largos**. Lo mismo con la data class **Etiqueta**.

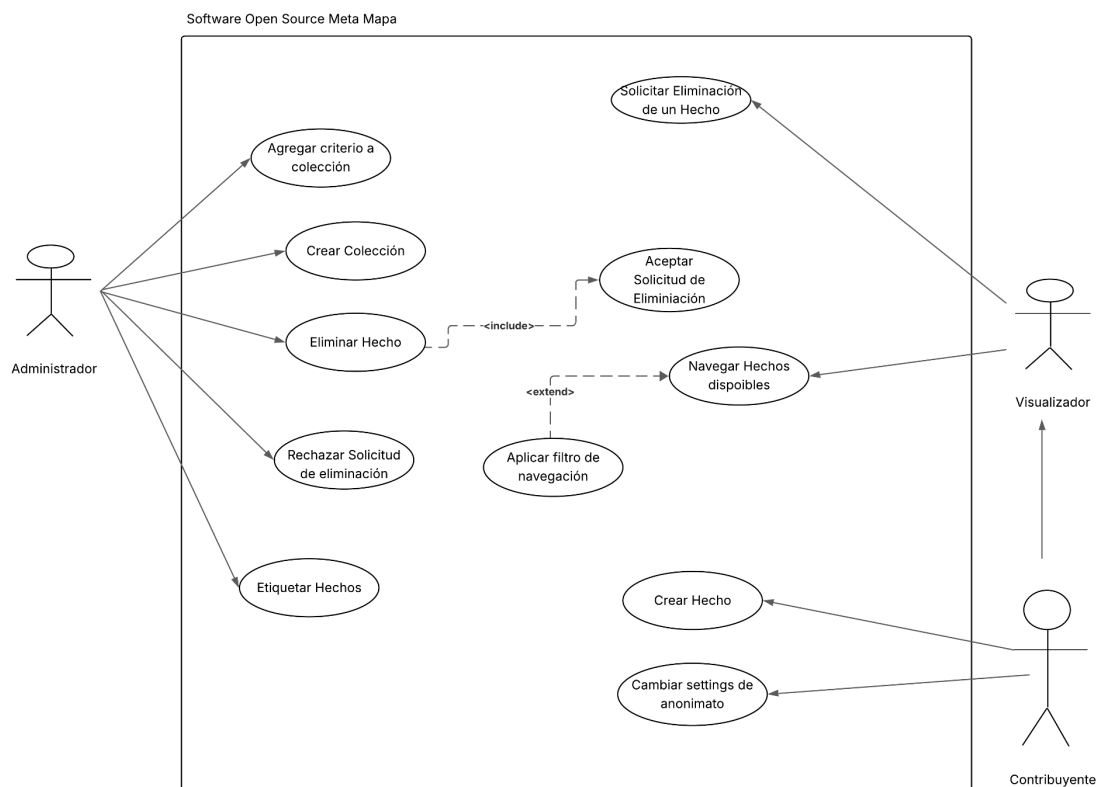
- **Colecciones:** conjuntos de hechos organizados bajo un título y descripción, creados y gestionados por administradores. Son públicas y no pueden ser editadas ni eliminadas manualmente.

Para implementar las colecciones nosotros decidimos crear una clase genérica que contenga todos los atributos que indica la consigna, incluyendo los criterios de pertenencia en forma de lista para poder aplicar todos los que se quiera. La clase **Coleccion** permite quitar o agregar criterios, además de realizar búsquedas de hechos de forma particular o mediante filtros.

- **Criterios:** Las colecciones también contarán con un **criterio de pertenencia** configurable, que dictará si un hecho pertenece o no a las mismas.

Para estos se aplicó el **Patrón Strategy**, ya que permite encapsular distintas formas de resolver el mismo problema en diferentes clases. En nuestro caso, el contexto sería la clase **Colección** y el **Criterio** representaría la estrategia como clase abstracta. Esto sucede ya que existen N estrategias concretas hijas, que son los diferentes criterios (por Categoría, Fecha, Lugar, etc.) y resuelven el mismo problema (determinar pertenencia). Esta implementación genera **mayor cohesión** a la clase Colección, **mayor mantenibilidad** debido a que el comportamiento por cada tipo de criterio es fácilmente localizable y **mayor extensibilidad** para incorporar nuevos criterios evitando métodos largos o código duplicado.

3. Diagrama General de Casos de Uso.



4. Implementación de los requerimientos.

Link al repositorio: <https://github.com/dds-utn/2025-tpa-ma-ma-grupo-05>